



UNIVERZITET U NIŠU
ELEKTRONSKI FAKULTET U
NIŠU



Master akademske studije
Računarstvo i informatika
Modul: Softversko inženjerstvo

Seminarski rad iz predmeta
Sistemi za upravljanje bazama podataka

Replikacija podataka (replication and mirroring) kod Firebird baze podataka

Profesor:

dr Aleksandar S. Stanimirović

Student:

Jefimija Stamenović 1820/24

Mart, 2026. godine

Sadržaj

1.	Uvod	4
1.1.	Definisanje problema: Konzistentnost vs Dostupnost	4
1.2.	Evolucija mehanizama u Firebird-u	5
1.3.	Cilj rada i metodologija	6
1.4.	Komparativna analiza: Preslikavanje vs Replikacija	7
2.	Implementacija i arhitektura preslikavanja	8
2.1.	Arhitektura i upis na nivou stranica	8
2.2.	Implementacija i upravljanje rezervnim datotekama	9
2.3.	Režimi rada: Manuelno nasuprot Automatskom preslikavanju	9
2.4.	Scenario oporavka i aktivacija rezervne kopije	10
2.5.	Ograničenja i izazovi u praksi	11
3.	Nativna binarna replikacija	12
3.1.	Koncept i arhitektura binarnih dnevnika promena	12
3.2.	Konfiguracija i parametri upravljanja sistemom	13
3.3.	Režimi rada: Sinhrona vs. Asinhrona replikacija	14
3.3.1.	Sinhroni režim	14
3.3.2.	Asinhroni režim	14
3.4.	Korišćenje sekundarnih servera za čitanje	15
4.	Logička replikacija zasnovana na trigerima	16
4.1.	Mehanizam hvatanja promena	16
4.1.1.	Kreiranje sistemskih tabela za logovanje promena	16
4.2.	Implementacija trigera za automatski upis u tabelu REPLICATION_LOG	17
4.3.	Mehanizam prenosa i izvršavanja na replici	18
4.4.	Strategije rešavanja konflikata	19
4.5.	Prednosti logičke replikacije	19
5.	Multi-Master konfiguracija i mrežne topologije	20
5.1.	Problem identifikatora: GUID vs. Generatori	20
5.2.	Topologije u Firebird okruženju	20
5.3.	Dvosmerna sinhronizacija i “eho” efekat	21
6.	Praktična implementacija	22
6.1.	Podešavanje sistema	22
6.2.	Primeri	27

7.	Zaključak i poređenje sa drugim sistemima.....	30
8.	Literatura	31

1. Uvod

U savremenom inženjerstvu informacionih sistema, obezbeđivanje neprekidnosti poslovanja (engl. *Business Continuity*) i otpornost na kritične otkaze (engl. *Disaster Recovery*) predstavlja temelj arhitekture svake robusne baze podataka. Kod Firebird-a, ovi ciljevi se ostvaruju kroz sinhronizovanu primenu dva mehanizma: **preslikavanja (mirroring/shadowing)** i **replikacije**.

Dok **preslikavanje** obezbeđuje fizičku redundansu na nivou stranica baze kako bi se neutralisao rizik od kvara hardvera, **replikacija** omogućava logičku distribuciju transakcija na geografski udaljene čvorove. Razlika je suštinska: **preslikavanje** čuva integritet fajla u realnom vremenu na nivou istog ili susednog skladišnog podsistema, dok **replikacija** čuva dostupnost servisa čak i u slučaju totalnog uništenja primarne serverske infrastrukture. Implementacija ovih tehnologija direktno utiče na parametre pouzdanosti koje zahtevaju moderni ERP i finansijski sistemi.

1.1. Definisanje problema: Konzistentnost vs Dostupnost

U distribuiranom okruženju, Firebird se suočava sa primarnim izazovom očuvanja ACID svojstava transakcija unutar mrežne infrastrukture. Replikacija u ovom sistemu ne predstavlja prosto kopiranje redova iz tabela, već prenos stanja baze koji mora striktno uvažiti specifičnosti multiverzione arhitekture (MGA). Ključni tehnički problem leži u sinhronizaciji stranica transakcionog inventara (engl. *TIP - Transaction Inventory Page*) i pratećih verzija podataka između čvorova. Ukoliko master čvor izvrši *commit* transakcije, a dođe do prekida komunikacije ka *slave* čvoru pre nego što se prenesu sve promene unutar baze, nastaje situacija u kojoj je transakcija na primarnom serveru **definitivno izvršena**, dok je na sekundarnom serveru ona i dalje u stanju „u toku” (**active**) ili čak nije ni vidljiva. Time se direktno narušava integritet baze podataka jer sekundarni čvor prestaje da bude verna kopija primarnog. Ovo zahteva implementaciju mehanizama koji će osigurati da sekundarna baza ostane u ispravnom stanju čak i nakon prekida u prenosu podataka.

Tehnički izbor režima rada direktno je diktiran **CAP teoremom** (engl. *Consistency, Availability, Partition tolerance*) koja navodi da je u distribuiranim sistemima nemoguće istovremeno u potpunosti zadovoljiti sva tri parametra. U kontekstu Firebird-a, administrator mora definisati prioritet između sledeća dva stanja:

- **Konzistentnost (engl. Consistency)** – u ovom stanju je osigurano da su *master* i *slave* čvorovi identični u svakom trenutku. Zahteva sinhronu upise što podrazumeva da *master* server neće potvrditi završetak transakcije korisniku dok ne dobije povratnu informaciju od *slave* servera da je podatak trajno upisan na disk. Iako garantuje maksimalnu tačnost, ovakav pristup povećava vreme odziva sistema i direktno zavisi od kvaliteta mreže.
- **Dostupnost (engl. Availability)** – u ovom stanju je fokus na brzini rada i kontinuitetu servisa bez obzira na kašnjenja u mreži. Koristi se asinhrona replikacija gde *master* server nastavlja sa obradom odmah nakon lokalnog upisa, dok se podaci na sekundarni čvor šalju u pozadini. Prednost je brzina odziva sistema, ali postoji rizik da poslednje transakcije ne budu vidljive na *slave* čvoru ukoliko dođe do fizičkog uništenja primarnog servera.

Razumevanje ovih principa je neophodno za pravilno projektovanje baze. Ukoliko se zahteva apsolutna preciznost podataka, bira se model koji favorizuje konzistentnost, dok se u sistemima gde je bitna brzina bira model koji favorizuje dostupnost. Firebird omogućava konfiguraciju ovih parametara kroz dnevnik promena (engl. *journal*) dajući administratoru mogućnost da optimizuje rad baze prema specifičnim zahtevima infrastrukture. Na taj način, sistem se prilagođava realnim uslovima operativnog rada i stabilnosti mreže, uzimajući u obzir balans između brzine izvršavanja transakcija i nivoa zaštite podataka.

U nastavku su data dva primera sa ciljem da se razume kada se koristi koji princip:

1. **Sistemi sa prioritetom konzistentnosti** bi se koristili kod finansijskih aplikacija ili knjiženja u realnom vremenu gde je sinhrona replikacija neophodna. Svaka transakcija mora biti potvrđena na oba čvora pre nego što se smatra završenom. Bilo kakvo nepoklapanje u stanjima računa, čak i u trajanju od nekoliko sekundi, dovelo bi do netačnih informacija o stanju sredstava što je u bankarskom sektoru nedopustivo.
2. **Sistemi sa prioritetom dostupnosti** bi se koristio kod sistema za prikupljanje podataka sa mernih uređaja (npr. očitavanje pritiska u gasovodu) gde je prioritet da sistem stalno prihvata podatke koji stižu sa njih. Ukoliko se zbog kratkotrajnog pada mreže prenos podataka ka rezervnom serveru uspori, to neće ugroziti validnost cele baze. Sa druge strane, zaustavljanje upisa novih podataka na masteru zbog čekanja na potvrdu mreže bi dovelo do gubitka merenja u realnom vremenu.

1.2. Evolucija mehanizama u Firebird-u

Istorijski razvoj Firebird RDBMS-a obeležila je duga zavisnost od rešenja trećih strana ili manualnih implementacija zasnovanih na kompleksnim SQL triggerima. Ovi rani sistemi, poznati kao **logička replikacija**, zahtevali su od programera da kreiraju dodatne tabele i trigere nad svakom tabelom čije promene treba pratiti. Iako je ovaj pristup nudio visok stepen fleksibilnosti, on je istovremeno uvodio značajno procesorsko opterećenje (engl. *CPU overhead*) i komplikovao održavanje metapodataka baze. Ovakav pristup je značajno otežavao održavanje sistema. Svaki put kada bi se izmenila struktura baze — na primer, doda se nova kolona ili izmeni tip podatka u tabeli, administrator bi morao ručno da ažurira i sve prateće trigere i skripte za replikaciju. U realnom radu, ovakvi manualni koraci su često dovodili do previda i grešaka, što je direktno ugrožavalo stabilnost sistema i usklađenost podataka između servera.

Glavna prekretnica nastaje pojavom verzije Firebird 4.0. Ova verzija je u standardnu distribuciju uvela nativnu binarnu replikaciju. Ovaj mehanizam donosi fundamentalnu izmenu u arhitekturi jer se proces više ne oslanja na SQL sloj i dodatne programske komande, već engine direktno presreće binarne promene unutar svojih dnevnika (logova). Ovakav pristup značajno smanjuje opterećenje procesora, jer se beleženje izmena obavlja istovremeno sa redovnim upisom podataka na disk, bez potrebe za dvostrukom obradom istih informacija. Eliminacija komplikovanih trigera je olakšalo održavanje baze, i poboljšalo stabilnost sistema, naročito u okruženjima gde ima veliki broj transakcija u sekundi.

Nativna replikacija je takođe uvela standardizovan način upravljanja segmentima logova, omogućavajući njihovo arhiviranje i prenos putem eksternih skripti. Ovo je administratorima obezbedilo moćan alat za kreiranje složenih topologija, uključujući kaskadnu replikaciju gde jedan *slave* server može služiti kao *master* za naredni nivo. Prethodna rešenja, poput IBReplicator-a ili FBReplicator-a, iako korisna, često su zahtevala zasebne licence ili specifična znanja koja nisu bila

univerzalno primenljiva. Uvođenjem nativnog rešenja, Firebird je postao konkurentan u odnosu na *enterprise* baze podataka koje nude standardno ugrađenu podršku za visoku dostupnost

Pored toga, evolucija sistema je obuhvatila i unapređenje mrežnih protokola koji podržavaju prenos ovih podataka. Savremene verzije koriste optimizovane algoritme za rukovanje replikacionim saobraćajem, što je od ključnog značaja za rad putem javnih komunikacionih kanala. Današnja arhitektura omogućava potpunu transparentnost procesa, pri čemu aplikativni sloj baze podataka ne mora imati informacije o postojanju replikacije sve dok se ne dogodi **kritični prekid rada**. Ovaj razvojni put, od **tradicionalnih trigger-baziranih rešenja** do automatizovane binarne sinhronizacije, potvrđuje zrelost Firebird projekta i njegovu sposobnost da odgovori na najzahtevnije izazove savremenog upravljanja podacima.

1.3. Cilj rada i metodologija

Primarni cilj ovog seminarskog rada je detaljna tehnička analiza mehanizama za obezbeđivanje redundanse unutar Firebird ekosistema kroz uporedni prikaz različitih strategija zaštite. Rad teži da razjasni unutrašnje procese koji se odvijaju prilikom sinhronizacije podataka, sa posebnim akcentom na specifičnosti Multi-Generational Architecture (MGA). Analiziraćemo kako verzionisanje zapisa utiče na stabilnost replikacionog kanala i koje su najbolje prakse za minimizaciju latencije. Cilj je pružiti **metodološki okvir** za konfigurisanje parametara koji direktno utiču na performanse celokupnog distribuiranog sistema.

Metodologija rada se zasniva na kombinaciji teorijske analize dokumentacije i praktične implementacije. Prvi deo seminarskog rada fokusira se na fizički nivo zaštite (**preslikavanje**), gde se ispituje ponašanje baze prilikom simulacije otkaza primarnog skladišnog medijuma. Biće testirana brzina automatskog prebacivanja na rezervni fajl i uticaj sinhronizacije potvrde upisa na performanse aplikacije. Ovaj deo rada koristiće standardne dijagnostičke alate za praćenje I/O operacija diska kako bi se kvantifikovalo dodatno opterećenje koje proces preslikavanja uvodi u sistem.

Drugi, složeniji deo rada obuhvata analizu logičke i binarne replikacije uvedene u novijim verzijama. Ovde će se ispitivati prenos **dnevnika promena** u asinhronom režimu, sa posebnim osvrtnom na rešavanje sukoba u podacima unutar sistema gde više servera ima pravo upisa. Poseban naglasak biće stavljen na primenu alata **nbackup**, koji je ključan za kreiranje **početne kopije baze** na drugom serveru bez prekida redovnog rada. Metodologija uključuje i simulaciju mrežnih prekida kako bi se testirala sposobnost sistema da nastavi prenos podataka od trenutka prekida, uz očuvanje potpune ispravnosti baze.

U završnom delu, rad se bavi postupcima za **prelazak na rezervni server** u slučaju kvara i **povratak na glavni sistem** nakon popravke. Analiziraće se koraci potrebni za preusmeravanje korisničkih programa na novu bazu i promenu statusa pomoćne baze u režim za redovan rad. Praktični rezultati biće prikazani kroz primere podešavanja i skripti koje se mogu direktno primeniti u stvarnim sistemima. Krajnji ishod rada biće jasan skup preporuka za administratore koji žele da podese Firebird bazu tako da ona bude maksimalno pouzdana i uvek dostupna za rad.

1.4. Komparativna analiza: Preslikavanje vs Replikacija

Izbor između preslikavanja i replikacije zavisi isključivo od toga šta želimo da zaštitimo. Kod preslikavanja, cilj je da baza nastavi sa radom ako se pokvari sam disk unutar servera. Kod replikacije, cilj je da ceo sistem preživi čak i ako cela lokacija (zgrada ili grad) ostane bez struje ili interneta. Glavna razlika je u tome što preslikavanje kopira sirove delove fajla na drugi disk, dok replikacija šalje zabeležene izmene podataka sa jednog servera na drugi putem mreže.

Tabela u nastavku sumira tehničke parametre koji direktno utiču na stabilnost i brzinu rada u oba scenarija:

Karakteristika	Preslikavanje (Mirroring)	Replikacija (Firebird 4.0)
Nivo operacije	Fizički (stranice baze - <i>pages</i>)	Logički (transakcije/žurnali)
Sinhronizacija	Isključivo sinhrona	Sinhrona ili asinhrona
Tipična distanca	Lokalna (isti server/storage)	Geografska (udaljeni serveri/Cloud)
Uticaj na performanse	Visok (čeka se potvrda oba upisa)	Podesiv (zavisi od moda rada)
Upotreba kopije	Kopija je neaktivna do otkaza	Replika može služiti za <i>read-only</i> upite
Glavni cilj	Otpornost na kvar hardvera diska	Otpornost na katastrofe i skalabilnost

Tabela 1: Uporedni prikaz preslikavanja i replikacije

Da bismo ovo razumeli u realnim uslovima, razmotrimo dva česta scenarija:

- **Lokalna zaštita podataka (Apoteka/Prodavnica)** – ovde je prioritet da se prodaja ne zaustavi ako disk otkaže. Koristi se preslikavanje na drugi fizički disk unutar istog kućišta. Pošto je sve lokalno, mrežna kašnjenja ne postoje, a sistem automatski prebacuje rad na rezervni disk bez potrebe da radnik na kasi išta primeti ili restartuje.
- **Udaljena zaštita podataka (Distributivni centar)** – Ako magacin u jednom gradu izgubi struju ili izbije požar, podaci moraju biti sigurni u drugom gradu. Ovde se koristi replikacija. Ako je veza brza, koristi se sinhrona replikacija da nijedna faktura ne bi bila izgubljena. Sa druge strane, ako je veza spora ili nestabilna, koristi se asinhroni režim kako rad u centrali ne bi trpeo zbog mrežnih zagušenja, uz svest o tome da će rezervna kopija kasniti nekoliko sekundi za glavnom.

2. Implementacija i arhitektura preslikavanja

Preslikavanje (engl. *Shadowing/Mirroring*) predstavlja Firebird-ov ugrađeni mehanizam za kreiranje identične kopije baze podataka u realnom vremenu. Za razliku od replikacije, koja prenosi promene putem dnevnika transakcija, preslikavanje direktno duplira svaku zapisanu stranicu baze na nivou fajl sistema. Ovaj proces je potpuno nevidljiv za korisničke aplikacije i odvija se direktno unutar samog *engine*-a baze podataka.

2.1. Arhitektura i upis na nivou stranica

Kada je u sistemu uključeno **preslikavanje**, *engine* modifikuje svoj osnovni ulazno-izlazni podsistem kako bi podržao dvostruki upis. Za razliku od standardnog rada gde se podaci šalju samo u .fdb fajl, ovde se uvodi stroga paralelna kontrola koja izvršava operacije na najnižem nivou fizičke organizacije podataka.

Kada korisnička aplikacija pošalje komandu za potvrdu transakcije (*commit*), Firebird ne vrši upis samo pojedinačnih promena, već manipuliše kompletnim stranicama koje su u memoriji označene kao izmenjene (*dirty pages*). *Engine* tada inicira dvostruku I/O operaciju. Važno je naglasiti da se ovaj proces ne oslanja na operativni sistem ili eksterne kontrolere diska, već sam Firebird upravlja slanjem identičnih paketa podataka ka primarnoj bazi (.fdb) i datoteci preslikavanja (.shd).

Osnova ovakvog načina funkcionisanja je stroga sinhronizacija. U praksi, nit (*thread*) koja obrađuje zahtev ulazi u stanje čekanja dok ne dobije povratnu informaciju da su podaci fizički zapisani na oba medijuma. Ovim se dobija:

- **Garancija identičnosti** – ako primarni disk potvrdi upis, a rezervni prijavi grešku (npr. kvar ili nedostatak prostora), transakcija se u manuelnom režimu smatra neuspešnom. Time se eliminiše svaka mogućnost da rezervna kopija zaostane za originalom.
- **Bezbednost podataka: stanovišta integriteta**, ovo je najviši nivo zaštite jer sprečava „delimičan upis”. Podaci su ili identično sačuvani na oba mesta, ili operacija uopšte nije trajno zabeležena.

Ovakav pristup, gde Firebird bazu posmatra kao niz stranica (obično od 4KB do 16KB), a ne kao skup tabela, pruža dodatnu sigurnost:

1. **Binarna podudarnost** – *Engine* ne mora da analizira SQL komande, već jednostavno kopira binarni sadržaj stranice. Čak i ako se promeni samo jedan podatak u ogromnoj tabeli, prepisuje se čitava stranica u oba fajla.
2. **Očuvanje strukture baze** – Budući da se preslikavaju apsolutno sve stranice, uključujući sistemske tabele, indekse i transakcioni inventar (**TIP**), rezervna datoteka je u svakom trenutku potpuno funkcionalna i spremna da postane glavna baza bez ikakve popravke.
3. **Otpornost na logičke greške** – Ovakav način rada osigurava da unutrašnja struktura baze (pokazivači i stabla indeksa) ostane identična na oba mesta, što je presudno za stabilnost sistema u trenutku kvara.

2.2. Implementacija i upravljanje rezervnim datotekama

Aktivacija preslikavanja u Firebird-u ne zahteva gašenje baze niti komplikovane administrativne alate jer se sve obavlja direktno putem SQL naredbi. Glavna komanda je `CREATE SHADOW` gde administrator dodeljuje redni broj „senci” (može ih biti više) i definiše putanju do datoteke.

Upravljanje ovim sistemom obuhvata nekoliko ključnih aspekata:

- **Fizička izolacija** podataka – osnovna preporuka je da se rezervna datoteka uvek nalazi na fizički drugom disku ili kontroleru. Ako se .shd fajl postavi na istu particiju gde je i .fdb baza, gubi se smisao zaštite od kvara hardvera jer bi otkaz tog diska uništio oba fajla istovremeno.
- **Skalabilnost redundanse** – Firebird omogućava kreiranje više rezervnih datoteka za jednu bazu (npr. `CREATE SHADOW 1...`, `CREATE SHADOW 2...`). U tom slučaju, sistem će vršiti paralelni upis na sve definisane lokacije, što dodatno povećava sigurnost, ali i linearno povećava opterećenje na procesor i diskovnu magistralu.
- **Praćenje rasta datoteke** – budući da se vrši binarno preslikavanje, datoteka sa nastavkom .shd raste identičnom brzinom kao i glavna baza. Administrator mora osigurati dovoljno slobodnog prostora na rezervnom medijumu jer bi u protivnom baza mogla prestati sa radom (u zavisnosti od izabranog režima).
- **Uklanjanje i redefinisavanje** – ukoliko je potrebno zameniti disk ili ukinuti zaštitu, koristi se naredba `DROP SHADOW`. Sistem tada trenutno prekida sinhroni upis i logički raskida vezu sa tom datotekom nakon čega se ona može bezbedno obrisati.

Sve informacije o definisanim kopijama čuvaju se unutar same baze, konkretno u sistemskoj tabeli `RDB$SHADOWS`. Ova tabela beleži broj kopije, putanju do fajla i izabrani režim rada. Zahvaljujući tome, Firebird pri svakom pokretanju servisa automatski proverava status rezervnih datoteka i pokušava da uspostavi sinhroni upis. Ukoliko neka datoteka nedostaje, sistem će reagovati u skladu sa definisanim pravilima (`AUTO` ili `MANUAL`), što značajno olakšava održavanje u realnim uslovima rada bez potrebe za stalnim nadzorom administratora.

2.3. Režimi rada: Manuelno nasuprot Automatskom preslikavanju

Prilikom kreiranja rezervne kopije naredbom `CREATE SHADOW`, administrator donosi ključnu odluku o prioritetima sistema u kritičnim trenucima. Ovaj izbor direktno utiče na to da li će sistem žrtvovati dostupnost radi integriteta podataka ili obrnuto. Na raspolaganju su dva režima rada:

1. **Manuelni režim (MANUAL)** – ovo je „strogi” režim gde je ispravnost kopije prioritet nad dostupnošću sistema. Ukoliko dođe do bilo kakve greške pri upisu u rezervni fajl (npr. disk se napuni ili dođe do kvara), primarna baza trenutno prestaje sa radom i izbacuje grešku. Ovaj pristup je neophodan u sistemima sa nultom tolerancijom na rizik, gde podaci ne smeju postojati bez svoje identične kopije ni na jednu milisekundu. Pošto je upis sinhronizovan, nit koja ne dobije potvrdu sa rezervnog diska smatra da transakcija nije bezbedna i odbija da je trajno završi čime se sprečava bilo kakvo odstupanje između dva fajla.
2. **Automatski režim (AUTO)** – u ovom režimu, Firebird favorizuje neprekidan rad sistema. Ukoliko upis u rezervni fajl ne uspe, *engine* ga automatski isključuje iz operativne upotrebe

(fajl dobija status "oboren") i nastavlja rad isključivo sa primarnom bazom. Korisnici ne osećaju prekid u radu, ali baza nastavlja da funkcioniše bez redundanse. Informacija o kvaru se beleži u **firebird.log** datoteci. Nakon što se otkloni kvar na disku, stara rezervna datoteka se više ne može koristiti jer je zaostala za glavnom bazom. Administrator mora ručno obrisati tu definiciju i kreirati novu rezervnu kopiju kako bi sistem ponovo bio zaštićen.

Izbor režima zavisi od poslovnih potreba. Ako je baza deo kritičnog finansijskog sistema, bira se **MANUAL** jer je bolje da sistem stane nego da postoji i najmanja sumnja u postojanje kopije. U maloprodaji ili logističkim centrima češće se bira **AUTO**, jer je trošak zaustavljanja rada svih radnika obično veći od rizika da sistem neko vreme radi sa samo jednim ispravnim diskom do dolaska serviser.

2.4. Scenario oporavka i aktivacija rezervne kopije

Jedna od najznačajnijih funkcionalnosti ovog mehanizma je sposobnost sistema da rezervnu datoteku pretvori u primarnu bazu bez ikakve intervencije administratora ili prekida rada aplikacija. Ovaj proces, poznat kao **promocija**, odvija se kroz nekoliko koraka:

- **Detekcija otkaza u realnom vremenu** – ako primarni fajl (.fdb) postane nedostupan — bilo zbog fizičkog kvara diska, oštećenja strukture fajla ili gubitka putanje — engine detektuje prekid I/O operacija. Budući da je svaka transakcija do tog trenutka bila sinhronizovana, sistem ima apsolutnu garanciju da je rezervni fajl (.shd) spreman da preuzme rad.
- **Automatsko preusmeravanje i preuzimanje uloge** – onog trenutka kada primarni fajl postane nedostupan, engine automatski preusmerava sve aktivne i dolazne operacije na rezervnu datoteku. Rezervni fajl istog trenutka prestaje da bude „senka” i dobija status glavne baze. Ovaj prelaz je za same korisničke programe gotovo neprimetan; aplikacije koje su povezane na bazu nastavljaju sa radom, uz eventualnu milisekundnu latenciju tokom koje sistem vrši internu rekonfiguraciju putanja.

Sintaksa kojom se omogućava ovakav stepen automatizacije je direktna i obuhvata definisanje automatskog režima rada:

Izvorni kod: Kreiranje shadow-a

1 CREATE SHADOW 1 AUTO 'D:\Mirror\naziv_baze.shd'

Ovakav scenario oporavka eliminiše potrebu za dugotrajnim procesom vraćanja podataka iz rezervnih kopija (*backup-a*), koji u velikim sistemima može trajati satima. Ovde je vreme oporavka (**RTO - Recovery Time Objective**) svedeno na nulu, što ovaj mehanizam čini idealnim za sisteme koji zahtevaju rad 24/7.

Da bismo proverili da li je naša kopija zaista i napravljen, možemo izvršiti sledeći SQL upit

Izvorni kod: Provera postojanja shadow-a
--

2 SELECT RDB\$SHADOW_NUMBER, RDB\$FILE_NAME, RDB\$FILE_FLAGS
3 FROM RDB\$FILES
4 WHERE RDB\$SHADOW_NUMBER > 0;

Na kraju, proverićemo i da li shadow zaista i preslikava sve akcije sa originalne baze:

Izvorni kod: Provera rada shadow-a
UPDATE EMPLOYEE SET PHONE_EXT = '999' WHERE EMP_NO = 2; COMMIT;

Ukoliko želimo da uklonimo shadow, to radimo komandom:

Izvorni kod br 1:
DROP SHADOW 1; -- shadow number

2.5. Ograničenja i izazovi u praksi

Iako preslikavanje zvuči kao idealno rešenje zbog svoje jednostavnosti i brzine oporavka, ono nosi određena ograničenja koja ga čine nepogodnim za moderne, fizički udaljene sisteme i *Cloud* arhitekture. Glavni izazovi su:

- **Pad performansi usled sinhronizacije (I/O Penalty)** – budući da je upis uvek sinhronizovan, ukupna brzina baze je ograničena brzinom sporijeg diska u paru. Svaka operacija izmene podataka (INSERT, UPDATE, DELETE) mora da sačeka potvrdu sa obe lokacije. Ako se rezervni fajl nalazi na mrežnom drajvu ili sporijem eksternom disku, korisnici će osetiti značajno usporavanje u radu aplikacije.
- **Fizička zavisnost** – rezervni fajl mora imati identičnu unutrašnju strukturu (**ODS - On-Disk Structure**) i verziju Firebird baze kao i glavni fajl. To znači da je nemoguće kombinovati različite operativne sisteme (npr. primarna baza na Windows-u, a rezervna na Linux-u) ili različite verzije servera unutar ovog mehanizma.
- **Nedostatak selektivnosti ("Sve ili ništa")** – preslikavanje ne dozvoljava administratoru da izabere samo kritične tabele za čuvanje. Sistem uvek kopira apsolutno sve stranice baze, uključujući indekse, sistemske tabele, pa čak i privremene podatke. Ovo rezultira duplim utroškom prostora na diskovima što kod baza od nekoliko stotina gigabajata može predstavljati značajan trošak.
- **Geografska ograničenja i osetljivost na mrežu** – ovaj mehanizam je projektovan isključivo za rad unutar iste lokalne mreže (LAN) ili na različitim diskovima istog servera. Za udaljene lokacije ili rad u oblaku (*Cloud*), mrežni prekidi ili variranja u brzini interneta bi u sinhronom režimu konstantno blokirali rad glavne baze. Zbog toga je ovaj mehanizam nepraktičan za udaljenu zaštitu podataka, jer mrežna kašnjenja postaju usko grlo za sve korisnike sistema.
- **Odsustvo raspodele opterećenja** – za razliku od replikacije gde rezervna kopija može služiti za čitanje izveštaja i rasterećenje glavne baze, rezervni fajl kod preslikavanja predstavlja **isključivo pasivan resurs**. On ne može biti otvoren za čitanje niti za bilo kakvu analizu dokle god se nalazi u statusu senke. To znači da hardverski potencijal rezervnog servera ostaje neiskorišćen sve do trenutka eventualnog kvara primarnog sistema.

3. Nativna binarna replikacija

Uvođenjem verzije 4.0, Firebird je dobio direktnu, ugrađenu podršku za replikaciju, što je značajno unapredilo način na koji se podaci distribuiraju između više servera. Pre ove verzije, administratori su bili prinuđeni da koriste eksterne alate ili da ručno pišu kompleksne trigere nad svakom tabelom kako bi pratili promene. Takav pristup je bio težak za održavanje i značajno je usporavao svaku transakciju, jer je baza morala da izvršava dodatnu logiku pri svakom upisu.

3.1. Koncept i arhitektura binarnih dnevnika promena

Osnova novog sistema replikacije u Firebird 4.0 je **Replikacioni dnevnik** (engl. *Replication Log*). Za razliku od prethodnih verzija, gde se radilo na logičkom nivou, u ovoj verziji, Firebird pri svakoj promeni tj. pri naredbama INSERT, UPDATE, DELETE, generiše binarne podatke koji opisuju modifikaciju na nivou stranice baze. Ovim se postiže maksimalna brzina jer sistem ne mora ponovo da interpretira SQL naredbe na rezervnom serveru. Ovaj mehanizam se oslanja na dva ključna procesa:

- **Generisanje replikacionih segmenata** – promene se ne šalju pojedinačno jer bi to opteretilo mrežni saobraćaj, već se one akumuliraju u datoteke određene veličine koje nazivamo segmentima. Svaki segment nosi svoj jedinstveni redni broj (engl. *sequence number*). Ovakva organizacija je od presudnog značaja za stabilnost sistema jer ako dođe do prekida mrežne veze, replikacija će, po uspostavljanju kontakta odmah nastaviti rad tačno od onog rednog broja segmenta na kojem je stala. Ovim se eliminiše rizik od preskakanja podataka ili potrebe za ponovnom inicijalizacijom cele baze.
- **Uloga procesa za prenos i reprodukciju (Replication Server)** – unutar engine-a postoji namenski podsistem zadužen za upravljanje ovim segmentima. On čita podatke iz dnevnika, pakuje ih u mrežne pakete i šalje ih ka ciljnom (*target* ili *slave*) serveru. Na strani primaoca, specijalizovani proces prihvata ove pakete i vrši njihovu **reprodukciju** (engl. *replay*) direktno u kopiju baze. Ključno je to što se strogo poštuje redosled transakcija; promena se na rezervnom serveru može izvršiti samo ako su sve prethodne transakcije uspešno primenjene, čime se čuva integritet podataka.

Glavna prednost replikacije u odnosu na preslikavanje je u tome što preslikavanje zahteva da oba diska budu dostupna u istom trenutku, dok kod replikacije postoji dnevnik promena koji omogućava „odloženo” slanje. Ako rezervni server bude ugašen sat vremena, primarna baza će nastaviti da gomila segmente u dnevnik i poslaće ih „u paketu” čim se rezervni server vrati na mrežu, bez ikakvog zastoja za krajnje korisnike na glavnoj lokaciji.

3.2. Konfiguracija i parametri upravljanja sistemom

Za razliku od preslikavanja koje se oslanja na SQL komande, nativna replikacija u Firebird-u se oslanja na eksterni konfiguracioni fajl **replication.conf**. Ovakav pristup omogućava administratoru da precizno uskladi rad replikacionog engine-a sa hardverskim mogućnostima servera, bez potrebe za izmenama unutar same baze.

Ključne stavke koje direktno utiču na stabilnost i brzinu prenosa podataka su:

- **Velicina bafera (buffer_size)** – Ovaj parametar određuje koliko će RAM memorije biti rezervisano za privremeno čuvanje replikacionih podataka pre njihovog trajnog zapisa na disk ili slanja kroz mrežu. U sistemima sa velikim brojem transakcija, veći bafer (u našem primeru postavljen na 64 MB) sprečava zagušenje glavnog procesa jer korisnik ne mora da čeka da se spori I/O procesi završe.
- **Lokacija žurnala (journal_directory)** – Predstavlja putanju na master serveru gde se čuvaju binarni segmenti promena. Za optimalan rad, ovaj direktorijum treba smestiti na najbrži dostupni medijum (SSD) jer brzina upisa u ove fajlove direktno diktira ukupnu propusnu moć.
- **Arhiviranje i transport (journal_archive_command)** – Firebird koristi automatizovan mehanizam za prenos popunjenih segmenata. Parametar journal_archive_directory definiše privremeno skladište, dok sama komanda za arhiviranje (u našem slučaju sistemska copy naredba) obezbeđuje da podaci bezbedno stignu na odredište. Dodatni parametar journal_archive_timeout služi kao mehanizam za forsiranje slanja podataka čak i ako segment nije potpuno popunjen čime se smanjuje kašnjenje (latency) između dva servera.

Ispod je primer kako bi izgledao deo konfiguracionog fajla **replication.conf** za produkcionu serveru gde je fokus na performansama i sigurnosti:

Izvorni kod: Podešavanje replication.conf fajla

```

5  database = putanja do baze\naziv_baze.fdb
6  {
7      # Velicina bafera od 64MB za teze transakcije
8      buffer_size = 67108864
9
10     # Lokacija journal fajlova na brzom SSD disku
11     journal_directory = D:\Baze\REPLICATION\Journal
12
13     # Arhiviranje segmenata
14     journal_archive_directory = D:\Baze\REPLICATION\Archive
15
16     # Komanda koja se izvršava kada se segment popuni
17     journal_archive_command = "copy $(pathname) $(archivepathname)"
18
19     # Timeout arhiviranja u sekundama
20     journal_archive_timeout = 10
21 }
```

3.3. Režimi rada: Sinhrona vs. Asinhrona replikacija

Firebird 4.0 omogućava administratoru da precizno definiše balans između bezbednosti podataka i brzine odziva aplikacije. Postoje dva režima: sinhroni i asinhroni, a izbor zavisi prvenstveno od kvaliteta mrežne infrastrukture i kritičnosti podataka koji se obrađuju.

3.3.1. Sinhroni režim

U sinhronom režimu (engl. *Synchronous Mode*), kada korisnik izvrši *commit*, master server ne vraća potvrdu odmah. On prvo šalje binarni log sekundarnom (*replica*) serveru i čeka povratni signal da je podatak fizički zapisan na drugoj strani. Tek nakon te potvrde, aplikacija dobija informaciju da je transakcija uspešna.

Glavna prednost ovog režima je **nulti gubitak podataka (engl. Zero Data Loss)** jer je zagarantovana apsolutna identičnost između mastera i replike u svakom trenutku. Čak i u slučaju katastrofalnog kvara master servera (npr. požar u data-centru), poslednja potvrđena transakcija je sigurno sačuvana na rezervnoj lokaciji.

Međutim, ograničenje je **mrežna latencija** jer svaka operacija upisa direktno zavisi od brzine mreže. Na sporim ili nestabilnim vezama, vreme odziva aplikacije drastično raste, jer korisnik mora da čeka "povratno putovanje" (*round-trip*) mrežnog paketa.

3.3.2. Asinhroni režim

Kod asinhronog režima (engl. *Asynchronous Mode*), prioritet je maksimalna brzina rada lokalnih korisnika. Master server upisuje promenu u svoj lokalni dnevnik (*journal*) i odmah javlja aplikaciji da je transakcija uspešno završena, ne čekajući potvrdu sa druge strane. Slanje podataka na sekundarni server odvija se u pozadini kao potpuno nezavisan proces, što u praksi dovodi do sledećeg niza povezanih događaja:

- 1) **Formiranje reda čekanja (Queue)** – ukoliko je mrežna veza privremeno spora ili zagušena, binarni logovi ne mogu biti poslani istom brzinom kojom nastaju. Zbog toga se na master serveru formira **red čekanja** u vidu nagomilanih fajlova na disku. Čim se propusna moć mreže popravi, sistem automatski ubrzava prenos kako bi "sustigao" stanje na primarnoj bazi.
- 2) **Pojava replikacionog kašnjenja (Replication Lag)** – direktna posledica ovog reda čekanja je **replikaciono kašnjenje**, odnosno vremenski razmak između trenutka upisa na masteru i trenutka kada taj isti podatak postane vidljiv na replici.
- 3) **Definisanje tačke oporavka (RPO)** – upravo to kašnjenje u teoriji zaštite podataka definiše **Ciljnu tačku oporavka (RPO - Recovery Point Objective)**. To je parametar koji administratoru govori koliko bi podataka sistem mogao da izgubi u slučaju totalne havarije master servera pre nego što svi nagomilani logovi iz reda čekanja stignu do replike.

3.4. Korišćenje sekundarnih servera za čitanje

Najveća praktična i ekonomska korist od native replikacije u Firebird-u 4.0 je mogućnost aktivnog korišćenja rezervne baze podataka dok primarni server nesmetano obavlja upise. Ovaj proces se u stručnoj literaturi naziva **rasterećenje primarnog servera** (engl. *offloading*).

Ovaj mehanizam funkcioniše kroz dva ključna aspekta:

- **Rasterećenje SELECT upita (engl. Query Offloading)** – u sistemima sa velikim brojem transakcija, primarni server je često maksimalno opterećen procesima unosa i izmene podataka (npr. stalno kreiranje fiskalnih računa ili naloga za prenos). Izvršavanje teških i složenih SQL upita za mesečne izveštaje ili analitiku na takvom serveru bi izazvalo zagušenje i usporilo rad korisnika. Uvođenjem replikacije, svi analitički upiti se preusmeravaju na sekundarni server. Na taj način, master server se bavi isključivo upisom, dok rezervni server služi kao platforma za izveštavanje i poslovnu inteligenciju (BI).
- **Hibridni režim rada (Read-Only Mode)** – sekundarna baza je za krajnje korisnike i aplikacije u režimu samo za čitanje (engl. *Read-Only*). Međutim, na nivou samog *engine*-a, ona je veoma aktivna jer neprestano "konzumira" binarne segmente koji stižu od mastera. Zahvaljujući asinhronom prenosu, podaci u izveštajima su skoro u realnom vremenu — sa zakašnjenjem od svega nekoliko milisekundi ili sekundi, a da pritom nijedan zahtevan upit ne može da blokira ili uspori unos podataka na primarnoj lokaciji.

Važno je napomenuti da Firebird 4.0 omogućava da se konekcije na repliku definišu kao *Read-Only* na nivou same baze podataka. Čak i ako aplikacija greškom pokuša da izvrši UPDATE nad replikom, server će odbiti transakciju, čime se garantuje da replika ostaje verna kopija mastera.

4. Logička replikacija zasnovana na trigerima

Za razliku od binarne replikacije koja operiše nad stranicama baze, logička replikacija se oslanja na SQL trigere koji presreću DML operacije (INSERT, UPDATE, DELETE) u trenutku njihovog izvršavanja. Ovaj pristup se ne bavi time *kako* je podatak zapisan na disk, već *šta* se u podatku promenilo.

4.1. Mehanizam hvatanja promena

Da bi sistem znao koje podatke treba da prenese na drugi server, mora postojati pouzdan mehanizam za beleženje istorije promena unutar same baze podataka. Ovaj proces se u modernom inženjerstvu baza podataka naziva **CDC (engl. Change Data Capture)**

U Firebird-u se ovaj mehanizam najčešće implementira kroz sledeće korake:

- **Namenske tabelle žurnala (engl. Audit/Log tables)** – administrator kreira posebne tabelle u koje će se upisivati svaka promena. Ove tabelle obično čuvaju informaciju o tome koja je tabela menjana, koji je primarni ključ obuhvaćen, koji je tip operacije (npr. 'U' za Update), kao i same podatke (stare i nove vrednosti).
- **Izvršavanje trigera (engl. Trigger Execution)** – nad svakom tabelom koju želimo da replicujemo, postavljamo trigere AFTER INSERT OR UPDATE OR DELETE. U trenutku kada korisnik sačuva promenu, trigger se aktivira i automatski kopira te podatke u tabelu žurnala. Jako je bitno napomenuti da se ova akcija odvija unutar iste transakcije kao i sama promena. Ako primarna transakcija padne (Rollback) i zapis u žurnalu se poništava, pa se time garantuje konzistentnost.
- **Identifikacija redosleda (engl. Sequencing)** – svaki zapis u žurnalu dobija svoj jedinstveni, rastući identifikator (obično preko SEQUENCE ili GENERATOR objekta). Ovo je ključno kako bi se promene na ciljnoj bazi mogle primeniti tačno onim redosledom kojim su nastale na izvornom server

Postavlja se pitanje, „Zašto koristiti CDC umesto binarne replikacije?“, a odgovor leži u tome što je glavna prednost ovog pristupa selektivnost. Dok binarna replikacija kopira celu bazu, CDC omogućava da izaberemo samo ključne tabelle koje želimo da šaljemo na drugi server čime se drastično štedi mrežni saobraćaj i prostor na disku.

4.1.1. Kreiranje sistemskih tabela za logovanje promena

Prvi korak u implementaciji logičke replikacije je uvođenje namenskih pomoćnih tabela, koje se u arhitekturi distribuiranih sistema često nazivaju **REPLICATION_LOG** ili **OUTBOX** tabelle. Njihova uloga je da služe kao privremeno skladište (bafer) za sve promene koje tek treba da budu poslate na drugi čvor. Da bi se promena uspešno rekonstruisala na udaljenom serveru, svaka stavka u ovoj tabeli mora da sadrži sledeći minimum informacija:

- **Identifikator promene (LOG_ID)** – jedinstveni, rastući broj (generisan putem SEQUENCE objekta) koji garantuje da će se promene na ciljnoj bazi izvršavati strogo onim redosledom kojim su nastale na izvornom serveru.
- **Metapodaci o operaciji** – naziv tabelle nad kojom je izvršena DML operacija i oznaka tipa promene (obično 'I' za *Insert*, 'U' za *Update*, 'D' za *Delete*).

- **Identifikator podatka (PK_VALUE)** – primarni ključ menjanog zapisa. Ovo omogućava replikacionom agentu da tačno zna koji red treba ažurirati ili obrisati na drugoj strani.
- **Vreme (TIMESTAMP)** – precizno vreme izvršenja transakcije, korisno za rešavanje eventualnih konflikata i praćenje kašnjenja (*lag-a*).
- **Sadržaj promene (Payload)** – podaci koji su predmet replikacije. Obično se pakuju u BLOB polje u obliku JSON ili XML formata. Ovakav pristup omogućava fleksibilnost – ista tabela žurnala može da skladišti promene iz potpuno različitih tabela (npr. i račune i šifarnik artikala) bez menjanja svoje strukture

Primer definicije ovakve tabele u Firebird-u:

Izvorni kod br 3:	
22	CREATE TABLE REPLICATION_LOG (
23	LOG_ID BIGINT NOT NULL PRIMARY KEY,
24	TABLE_NAME VARCHAR(31) NOT NULL,
25	OPERATION_TYPE CHAR(1) NOT NULL, -- 'I', 'U', 'D'
26	RECORD_ID VARCHAR(128) NOT NULL, -- Primarni ključ originalnog zapisa
27	DATA_PAYLOAD BLOB SUB_TYPE TEXT, -- Podaci u JSON formatu
28	CREATED_AT TIMESTAMP DEFAULT CURRENT_TIMESTAMP
29	);

Iako ove tabele omogućavaju veliku kontrolu, administrator mora voditi računa o njihovom čišćenju. Nakon što eksterni proces uspešno prebaci podatke na drugi server, zapisi iz ove tabele se moraju brisati kako bi se sprečilo nepotrebno trošenje prostora na disku i usporavanje baze.

4.2. Implementacija trigera za automatski upis u tabelu REPLICATION_LOG

Trigeri su izvršni blokovi koda koji se automatski pokreću pri svakoj promeni podataka. Za potrebe replikacije, najbolje je koristiti AFTER trigere, jer oni garantuju da će se zapis u dnevnik promena izvršiti tek nakon što je osnovna operacija uspešno prošla sve provere integriteta.

Pretpostavimo da želimo da replicujemo promene nad šifarnikom artikala. Triger bi izgledao ovako:

Izvorni kod br 4:	
30	CREATE OR ALTER TRIGGER TR_ARTIKLI_REPLICATE
31	AFTER INSERT OR UPDATE OR DELETE ON ARTIKLI
32	AS
33	DECLARE VARIABLE log_type CHAR(1);
34	BEGIN
35	/* 1. Određivanje tipa operacije */
36	IF (INSERTING) THEN log_type = 'I';
37	ELSE IF (UPDATING) THEN log_type = 'U';
38	ELSE IF (DELETING) THEN log_type = 'D';

```

39
40      /* 2. Upis u sistemsku tabelu REPLICATION_LOG */
41      INSERT INTO REPLICATION_LOG (
42          LOG_ID,
43          TABLE_NAME,
44          OPERATION_TYPE,
45          RECORD_ID,
46          DATA_PAYLOAD
47      ) VALUES (
48          NEXT VALUE FOR SEQ_REPLICATION_LOG,
49          'ARTIKLI',
50          :log_type,
51          COALESCE(NEW.ID, OLD.ID), -- Uzima ID novog ili starog
zapisa
52          (SELECT json_object FROM ( -- Primer pakovanja u JSON
(Firebird 3.0+)
53              SELECT
54                  NEW.NAZIV AS "naziv",
55                  NEW.CENA AS "cena",
56                  NEW.GRUPA_ID AS "grupa"
57              FROM RDB$DATABASE
58          ))
59      );
60  END

```

4.3. Mehanizam prenosa i izvršavanja na replici

Samo postojanje zapisa u tabeli žurnala nije dovoljno za samu replikaciju; potreban je eksterni mehanizam koji će te podatke fizički prebaciti na odredište. Za razliku od binarne replikacije gde taj posao obavlja sam Firebird *engine*, kod logičke replikacije tu ulogu preuzima **replikacioni agent**. To je nezavisan softverski servis koji u pravilnim vremenskim intervalima (npr. svake sekunde) skenira tabelu žurnala na izvornom serveru.

Kada agent identifikuje nove, neposlate zapise, on ih putem mreže prosleđuje do ciljne baze podataka. Tamo se proces završava **izvršavanjem SQL komandi** koje rekonstruišu originalnu promenu: ako je u žurnalu zabeležen *Insert* sa određenim podacima, agent će nad replikom izvršiti istovetnu operaciju. Tek nakon što ciljni server potvrdi uspešan upis, agent na izvornom serveru označava te zapise kao obrađene ili ih briše, čime se sprečava dvostruko slanje i oslobađa prostor na disku. Ovakva arhitektura omogućava da sistem funkcioniše čak i u uslovima povremenih prekida internet veze, jer će agent jednostavno nastaviti slanje tamo gde je stao čim se komunikacija ponovo uspostavi.

4.4. Strategije rešavanja konflikata

U **Multi-Master** konfiguracijama, gde svaki čvor dozvoljava izmenu podataka, neminovan je rizik od istovremene promene istog zapisa na različitim lokacijama. Dok replikacioni agent ne razmeni te izmene, nastaje **privremeni raskorak u podacima**, što stvara direktan konflikt koji sistem mora da razreši određenom strategijom prioriteta.

Da bi se očuvao integritet, koriste se tri standardna pristupa:

- **Pravilo poslednjeg upisa (engl. LWW - Last Write Wins)** – ovo je najčešća strategija zasnovana na vremenskim oznakama (engl. *timestamp*). Sistem upoređuje precizno vreme nastanka promene na oba servera i zadržava onu koja je hronološki kasnija, dok se ranija promena odbacuje. Da bi ovo radilo, serveri moraju imati savršeno sinhronizovano vreme putem **NTP (Network Time Protocol)** protokola.
- **Prioritetni čvor (engl. Static Priority)** – u ovoj strategiji se unapred definiše hijerarhija servera. Ako dođe do konflikta, izmena koja potiče sa "glavnog" servera (npr. Centrala) uvek ima prednost nad izmenom sa "sporednog" servera (npr. Poslovnica), bez obzira na to koja je promena vremenski prva nastala. Ovo je veoma robusno rešenje za hijerarhijske organizacije.
- **Logika aplikativnog nivoa i ručna revizija** – kod kritičnih podataka (npr. finansijske transakcije ili stanje zaliha), automatika može biti opasna. U tim slučajevima, replikacioni agent ne briše nijednu verziju, već konfliktni zapis prebacuje u tabelu za reviziju (engl. *Audit/Conflict table*). Sistem tada obaveštava administratora koji ručno pregleda obe verzije i donosi konačnu odluku o tome koji podatak je ispravan.

4.5. Prednosti logičke replikacije

Iako je logička replikacija znatno zahtevnija za implementaciju i održavanje, ona nudi rešenja koja nativni, binarni mehanizmi po svojoj prirodi ne mogu da obezbede. Jedna od njenih najznačajnijih prednosti je mogućnost povezivanja servera sa potpuno različitim verzijama baze (engl. **Cross-version Replication**). Ovo je često jedini pouzdan način da se podaci sinhronizuju između zastarelih instalacija, poput Firebird 2.5, i modernih verzija 4.0 ili 5.0. Budući da se putem SQL upita prenose same vrednosti polja, a ne fizičke stranice baze, verzija strukture na disku (ODS – *On-Disk Structure*) prestaje da bude prepreka za razmenu informacija.

Pored toga, ovaj pristup omogućava preciznu kontrolu nad opsegom podataka kroz **parcijalnu replikaciju**. Dok nativni sistem uvek kopira kompletnu bazu, triggerima se može definisati da se prenose samo ključne tabele, poput računa i stavki, dok se lokalni šifarnici ili privremeni podaci potpuno ignorišu radi uštede mrežnih resursa. Dodatnu vrednost daje mogućnost **transformacije podataka** u samom trenutku prenosa, što dozvoljava sistemu da automatski vrši konverzije valuta, spajanje imena i prezimena u jedno polje ili prilagođavanje formata potrebama ciljnog servera. Ovakvi procesi ulaze u samu srž izazova kod distribuiranih sistema, gde fokus nije samo na kopiranju, već na inteligentnom protoku podataka kroz više čvorova istovremeno.

5. Multi-Master konfiguracija i mrežne topologije

Implementacija Multi-Master replikacije u Firebird okruženju, gde svaki čvor u mreži poseduje autonomiju da vrši izmene nad podacima, predstavlja jedan od najsloženijih izazova u radu sa distribuiranim bazama. Za razliku od binarne (Master-Slave) replikacije, gde je smer kretanja informacija strogo definisan, Multi-Master zahteva napredne mehanizme za očuvanje globalnog integriteta i sprečavanje logičkih grešaka koje nastaju usled istovremenih upisa na različitim geografskim lokacijama.

5.1. Problem identifikatora: GUID vs. Generatori

Prvi kritičan izazov distribuiranog sistema je očuvanje jedinstvenosti primarnih ključeva. Budući da klasični GENERATOR (Sequence) u Firebirdu operiše isključivo na nivou lokalne baze, bez svesti o drugim čvorovima, postoji visok rizik od kolizije — situacije u kojoj dva različita servera dodeljuju isti ID različitim zapisima. U praksi se ovaj problem prevazilazi na dva načina:

- **Upotreba UUID/GUID-a:** Funkcija GEN_UUID() obezbeđuje globalnu jedinstvenost bez potrebe za sinhronizacijom između servera. Ipak, glavni nedostatak ovog rešenja je fragmentacija indeksa; zbog nasumične prirode vrednosti, Firebird mora češće da reorganizuje stranice baze, što može uticati na performanse kod sistema sa ekstremno visokim brojem transakcija
- **Segmentacija BIGINT generatora:** Alternativni pristup je dodeljivanje specifičnog "koraka" ili opsega svakom čvoru. Na primer, konfiguracijom tako da Server 1 generiše isključivo neparne (1, 3, 5...), a Server 2 parne brojeve (2, 4, 6...), postiže se unikatnost uz zadržavanje brzine koju nude celobrojni tipovi podataka i sekvencijalni upis.

5.2. Topologije u Firebird okruženju

Izbor mrežne arhitekture direktno određuje putanje kojima se podaci kreću kroz sistem, ali i brzinu kojom će se podaci usaglasiti na svim čvorovima. U Firebird instalacijama, izbor topologije najčešće zavisi od hijerarhije poslovne organizacije i pouzdanosti mrežne infrastrukture. Postoje tri tipa topologije:

- **Zvezda (Star): Zvezda (engl. Star Topology)** – u ovom modelu, svi periferni čvorovi su povezani isključivo sa centralnim serverom (engl. *Hub*). Svaka promena nastala na periferiji prvo se šalje u centar, koji zatim vrši distribuciju te promene ostalim čvorovima. Glavni nedostatak ove topologije je postojanje **jedinstvene tačke otkaza** (engl. *Single Point of Failure*), a to znači ako centralni server postane nedostupan, cela mreža gubi mogućnost međusobne sinhronizacije, iako lokalni čvorovi nastavljaju sa radom.
- **Mreža (Mesh)** – ovaj model podrazumeva da je svaki čvor u sistemu direktno povezan sa svim ostalim učesnicima (engl. *Full Mesh*) ili barem sa većinom njih. Osnovna prednost je maksimalna otpornost na prekide veza, jer podatak može pronaći alternativnu putanju do odredišta čak i ako jedan ili više servera otkazu. Ipak, cena ovakve sigurnosti je ekstremna kompleksnost u upravljanju replikacionim agentima. Broj konekcija raste eksponencijalno sa svakim novim čvorom, a rešavanje konflikata postaje značajno teže jer ista promena može stići do cilja iz više različitih izvora u različito vreme.

- **Prsten (Ring)** u ovoj konfiguraciji, podaci putuju sekvencijalno, od jednog čvora do drugog, dok se krug ne zatvori. Iako je logički jednostavna za implementaciju, ova topologija se u modernim Firebird sistemima izuzetno retko koristi. Glavni razlog je njena niska tolerancija na greške; prekid komunikacije na samo jednoj tački ili ispadanje jednog servera iz mreže momentalno zaustavlja protok informacija za ceo sistem. Takođe, latencija je ovde najveća, jer promena nastala na početku prstena mora proći kroz sve posredne čvorove pre nego što postane vidljiva na poslednjem serveru u lancu.

5.3. Dvosmerna sinhronizacija i “eho” efekat

Poseban nivo složenosti u svim mrežnim konfiguracijama predstavlja sprečavanje nastanka beskonačnih petlji, u literaturi poznatih kao **„eho” efekat**. Problem nastaje u trenutku kada Server A pošalje promenu Serveru B; ukoliko trigeri na Serveru B tu dolaznu promenu detektuju kao novu lokalnu aktivnost, oni će generisati zapis u sopstvenom logu i pokušati da tu istu informaciju vrate nazad izvornom serveru (Serveru A). Time se stvara zatvoreni krug koji dovodi do eksponencijalnog rasta bespotrebnih transakcija i potpunog zagušenja procesorskih resursa.

U modernim Firebird sistemima, a naročito od verzije **4.0**, ovaj problem se rešava uvođenjem provere konteksta transakcije. Korišćenjem sistemskih funkcija, triger na ciljnom serveru može u deliću sekunde da identifikuje da li je promenu napravio stvarni korisnik (putem aplikacije) ili je ona rezultat rada samog replikacionog agenta. Ključna implementacija ove logike oslanja se na proveru sesijske varijable:

Izvorni kod br 5:
<pre>61 IF (RDB\$GET_CONTEXT('USER_SESSION', 'REPLICATING') IS NULL) THEN ...</pre>

Ovaj uslov osigurava da će triger upisati zapis u dnevnik za replikaciju isključivo ako je promena nastala direktnom interakcijom sa bazom. Ukoliko je promenu uneo agent koji je prethodno postavio status REPLICATING, triger će tu operaciju ignorisati i neće je ponovo slati u krug. Na ovaj način se prekida lanac povratnih informacija, što je preduslov za stabilan rad bilo kog Multi-Master sistema.

6. Praktična implementacija

6.1. Podešavanje sistema

U ovom poglavlju prikazan je proces postavljanja **nativne sinhronne replikacije** i mehanizma preslikavanja identiteta (**Mapping**) na standardnom Firebird modelu EMPLOYEE.FDB.

Korak 1: Podešavanje database.conf fajla

U konfiguracionom fajlu **databases.conf** (lokacija: C:\Program Files\Firebird\Firebird_5_0), je potrebno kreirati dva aliasa: jedan za bazu koja će biti master (**SEMINARSKI_DB**) i drugi za bazu koja će služiti kao replika (**SEMINARSKI_DB_REPLIKA**). Master baza je izvorna baza u kojoj se vrše promene, dok je slave (Replika) baza odredište na koje se te promene automatski prenose.

Izvorni kod: Backup baze

```
62 SEMINARSKI_DB = C:\Program
   Files\Firebird\Firebird_5_0\examples\empbuild\EMPLOYEE.fdb
63 {
64     RemoteAccess = true
65     DefaultDbCachePages = 256
66 }
67
68 SEMINARSKI_DB_REPLIKA = D:\Baze\SEMINARSKI_DB_REPLIKA.fdb
69 {
70     RemoteAccess = true
71     DefaultDbCachePages = 256
72 }
```

Korak 2: Pravljenje rezervne kopije i kreiranje rezervne baze

Replikacija u Firebird sistemu se ne inicijalizuje automatski, pa je potrebno manuelno kreirati identičan „klon” originalne, odnosno produkcione baze podataka. Ovaj proces se naziva *Initial Seeding* i postizemo ga upotrebom standardnog gbak alata za backup i restore. Tokom ovog koraka, posebna pažnja je posvećena čišćenju meta-podataka kako bi se izbegli konflikti sa starim replikacionim markerima.

Prvi podkorak je pravljenje .fbk fajla.

Izvorni kod: Backup baze

```
73 gbak -b -v -nod localhost:SEMINARSKI_DB D:\Baze\PRELAZNA_BAZA.fbk
   -user SYSDBA -pass masterkey
```

Nakon što je napravljena rezervna kopija, potrebno je „raspakovati” bazu na lokaciju predviđenu za repliku. Ključni parametar ovde je `-replica none`. On osigurava da nova baza (SEMINARSKI_DB_REPLIKA.fdb) bude kreirana kao čista, standardna baza podataka, čime se sprečava da ona pokuša da se ponaša kao Master ili da nasledi neispravna replikaciona podešavanja iz backupa.

Izvorni kod: Restore baze

```
74 gbak -rep -v -replica none D:\Baze\PRELAZNA_BAZA.fbk  
D:\Baze\SEMINARSKI_DB_REPLIKA.fdb -user SYSDBA -pass masterkey
```

Ovim postupkom dobijena je fizički identična, ali logički „čista” baza podataka na lokaciji D:\Baze, čime su ispunjeni svi preduslovi za uspostavljanje direktne sinhronne veze.

Korak 3: Uzimanje GUID-a master baze

Nakon uspešnog završetka restore-a baze, neophodno je utvrditi jedinstveni globalni identifikator (**GUID**) master baze. Ovaj podatak je potreban za dalja podešavanja jer služi kao jedinstveni ključ koji povezuje master bazu sa njenim dnevniciama promena i replikacionim tokovima.

Za dobijanje ovih informacija koristimo alat **gstat** sa parametrom **-h** (header) koji prikazuje metapodatke zaglavlja baze podataka.

Izvorni kod: Pregled metapodataka zaglavlja

```
75 gstat -h "C:\Program  
Files\Firebird\Firebird_5_0\examples\empbuild\employee.fdb" -user  
SYSDBA -password masterkey
```

Korak 4: Kreiranje foldera

Da bi sistem replikacije bio pouzdan i otporan na greške, neophodno je pripremiti namensku strukturu foldera na disku. Ovi folderi predstavljaju privremeno i trajno skladište za dnevnik promena baze podataka. Čak i u scenarijima sinhronne replikacije, posedovanje definisanih putanja za dnevnike promena omogućava *engine-u* da beleži sekvence transakcija što olakšava oporavak sistema nakon eventualnog pada servera ili mrežnog prekida.

Izvorni kod: Kreiranje foldera

```
76 mkdir D:\Baze\REPLICATION\Journal  
77 mkdir D:\Baze\REPLICATION\Archive
```

Korak 5: Podešavanje replication.conf

Na istoj lokaciji gde se nalazi database.conf, smešten je i konfiguracioni fajl **replication.conf**. Dok prvi fajl služi za definisanje aliasa, replication.conf služi za detaljno podešavanje samog replikacionog procesa, putanja do žurnala i povezivanje baze sa njenim jedinstvenim identifikatorom (GUID).

U delu za instancu replikacije, za parametar `source_guid` neophodno je uneti prethodno sačuvanu **DATABASE GUID** vrednost Master baze, čime se vrši autorizacija Slave čvora.

Izvorni kod: Podešavanje konfiguracionog fajla replication.conf

```

78 database = C:\Program
   Files\Firebird\Firebird_5_0\examples\empbuild\employee.fdb
79 {
80     journal_directory = D:\Baze\REPLICATION\Journal
81     journal_archive_directory = D:\Baze\REPLICATION\Archive
82     journal_archive_timeout = 10
83     sync_replica =
   SYSDBA:masterkey@localhost:D:\Baze\SEMINARSKI_DB_REPLIKA.fdb
84 }
85
86 database = D:\Baze\SEMINARSKI_DB_REPLIKA.fdb
87 {
88     journal_source_directory = D:\Baze\REPLICATION\Archive
89     source_guid = "{7FCEDA33-EDCD-487C-A372-DA9854EEA4C7}"
90 }

```

Značenje ključnih parametara:

- Journal Directory – putanja na kojoj server privremeno skladišti binarne podatke o promenama (segmente žurnala) pre nego što ih proces replikacije obradi.
- Journal Archive Directory – lokacija na kojoj se čuvaju trajni zapisi žurnala nakon što se njihova obrada u "Journal" folderu završi.
- Journal Archive Timeout – maksimalno vreme (u sekundama) čekanja pre nego što se trenutni segment žurnala proglasi spremnim za arhiviranje čak i ako nije popunjen do maksimalne veličine.
- Sync Replica – definiše konekcioni string ka Slave bazi. Uključuje korisničko ime (SYSDBA), lozinku i mrežnu putanju do odredišnog .fdb fajla.
- Journal Source Directory – parametar unutar *slave* instance koji govori bazi odakle treba da preuzima (čita) arhivirane žurnale koji stižu sa *master* servera.
- Source GUID – sigurnosni mehanizam koji osigurava da *slave* baza prihvata podatke isključivo od *master* baze sa tačno određenim GUID-om.

Korak 5: Restartovanje servisa Firebird

Nakon što su modifikovani fajlovi **databases.conf** i **replication.conf**, neophodno je restartovati Firebird servis (putem *Services* panela u Windows-u ili kucanjem **net stop FirebirdServerDefaultInstance** i **net start FirebirdServerDefaultInstance** u cmd-u). Ovaj korak je obavezan jer Firebird učitava parametre replikacije isključivo prilikom inicijalizacije servisa.

Korak 6: Uključivanje publikacije na masteru

Narednja faza je aktivacija publikacije na *master-u*. Publikacija je mehanizam koji određuje koje tabele i podaci će biti obuhvaćeni procesom replikacije. U ovom radu, korišćena je opcija **INCLUDE ALL** kako bi se osigurala potpuna sinhronizacija celokupnog modela **EMPLOYEE** baze.

Izvorni kod: Aktivacija publikacije

```
91 ALTER DATABASE INCLUDE ALL TO PUBLICATION;
92 -- commit
93 SELECT COUNT(*) FROM RDB$PUBLICATIONS; -- očekivana vrednost je 1
```

Korak 7: Podešavanje režima rada replike

U konfiguraciji *slave* čvora je potrebno definisati njegov režim rada. U arhitekturi replikacije, je veoma važno da sekundarna baza ostane u **Read-Only** režimu za sve eksterne aplikacije i korisnike. Time se sprečava "divergencija podataka" tj. pojava gde se podaci na *master-u* i *slave-u* razlikuju što bi nepovratno narušilo proces sinhronizacije.

Za ovu operaciju koristimo alat gfix, koji služi za održavanje i promenu operativnih parametara same baze.

Izvorni kod: Podešavanje režima rada na replikaciji

```
94 gfix -replica read_only -user SYSDBA -password masterkey
    localhost:SEMINARSKI_DB_REPLIKA
```

Da bismo bili sigurni da je baza u režimu read-only, izvršićemo SQL upit koji je dat ispod:

Izvorni kod: Provera režima rada na replici

```
95 SELECT
96     MON$REPLICA_MODE AS REPLICA_MOD,
97     CASE MON$REPLICA_MODE
98         WHEN 0 THEN 'Nije replika (master baza)'
99         WHEN 1 THEN 'Read-Only replika'
100        WHEN 2 THEN 'Read-Write replika'
101     END AS OPIS
102 FROM MON$DATABASE;
```

Korak 8: Podešavanje mapiranja na replici

Poslednji korak u podešavanju sigurnosnog protokola je kreiranje pravila za mapiranje korisnika (Mapping). Budući da replikacija podrazumeva prenos transakcija sa jednog čvora na drugi, *slave* baza mora imati definisano pravilo kojim se dolazne konekcije sa *mastera* tretiraju kao autorizovani zahtevi za promenu podataka. Ova komanda se izvršava na *slave* bazi i služi da "prevede" dolazni identitet u lokalni administrativni nalog (SYSDBA).

Izvorni kod: Podešavanje mapiranja

```
103 CREATE OR ALTER MAPPING REPL_MAP
104 USING PLUGIN Srlp
105 FROM ANY USER
106 TO USER SYSDBA;
```

Da bismo bili sigurni da je preslikavanje aktivno i da sistem ispravno prepoznaje replikacionog agenta kao SYSDBA korisnika, izvršićemo sledeći SQL upit:

Izvorni kod: Provera ispravnosti mapiranja

```
107 SELECT
108     RDB$MAP_NAME AS IME_PRESLIKAVANJA,
109     RDB$MAP_USING AS PLUGIN,
110     RDB$MAP_TO AS MAPIRANO_NA
111 FROM RDB$AUTH_MAPPING;
```

Provera putem tabele RDB\$AUTH_MAPPING potvrđuje da je sigurnosni mehanizam ispravno konfigurisan, čime se omogućava replikacionom agentu da sa SYSDBA privilegijama vrši upis čak i u bazu koja je u *read-only* režimu. Ključna prednost ovakvog mapiranja je u tome što Firebird engine automatski prepoznaje te transakcije kao eksterne, čime se matematički eliminiše rizik od cirkularne replikacije i beskonačnog vraćanja promena nazad na master server. Bez ovog koraka, agent ne bi imao potrebne dozvole za rad, ili bi sistem ušao u petlju koja bi ugrozila stabilnost oba čvora.

Korak 9: Verifikacija rada sistema

Nakon što su završena sva konfiguraciona podešavanja na nivou servera, baze podataka i sigurnosnih protokola, vrši se finalna verifikacija. Cilj ovog koraka je da se praktično potvrdi da se svaka transakcija izvršena na *master* bazi (SEMINARSKI_DB) trenutno i bez greške preslikava na *slave* bazu (SEMINARSKI_DB_REPLIKA).

Primer: Da bismo testirali sistem, unosimo novog zaposlenog u tabelu EMPLOYEE na Master bazi. Koristimo EMP_NO = 1000 kako bismo lakše identifikovali naš testni podatak.

Izvorni kod: Dodavanje podatka na master-u

```
112 INSERT INTO EMPLOYEE (EMP_NO, FIRST_NAME, LAST_NAME, PHONE_EXT,
113     HIRE_DATE, DEPT_NO, JOB_CODE, JOB_GRADE, JOB_COUNTRY, SALARY)
114 VALUES (1000, 'Replication', 'Test', '250', current_timestamp,
115     '621', 'Eng', 2, 'USA', 97500);
114 -- commit
```

Odmah nakon izvršenja *commit* komande na *master-u*, pristupamo *slave* bazi podataka i vršimo upit nad istom tabelom.

Izvorni kod: Provera unosa na replici

```
115 SELECT *
116 FROM EMPLOYEE
117 WHERE EMPLOYEE.EMP_NO = 1000
```

Zaključak:

Svaki COMMIT na masteru pokreće upis u *journal segment*. Replikacioni agent kontinuirano skenira *journal_archive_directory* i primjenjuje nove segmente na repliku. Novi red postaje vidljiv na replici u roku od nekoliko sekundi (zavisno od *journal_archive_timeout* parametra).

6.2. Primeri

U ovom poglavlju detaljno su analizirani dodatni praktični scenariji koji potvrđuju operativnu stabilnost sistema pri izmenama postojećih zapisa. Testiranje se fokusira na kritične operacije ažuriranja i brisanja podataka (Update i Delete), kroz koje se prati preciznost sinhronizacije i strogo očuvanje referencijalnog integriteta između master i slave čvora. Pored same manipulacije podacima, ovi primeri demonstriraju kako replikacioni mehanizam upravlja kompleksnim relacijama unutar baze, osiguravajući da sekundarni server u svakom trenutku ostane verna i konzistentna kopija primarnog sistema.

Primer 1: Ažuriranje podataka na master-u

Nakon uspešnog testiranja unosa, želimo da proverimo da li će raditi i ažuriranje (UPDATE) podataka. Ovaj test simulira realan scenario promene plate zaposlenog u *master* bazi gde sistem mora da prepozna izmenu specifičnog polja i tu informaciju prosledi na *slave* čvor.

Izvorni kod: Brisanje na masteru
<pre> 118 UPDATE EMPLOYEE 119 SET SALARY = 99500.00 120 WHERE EMP_NO = 1001; 121 --COMMIT </pre>

Sledeći korak je provera ažurnosti podataka na replici

Izvorni kod: Provera ažuriranja na replici
<pre> 122 SELECT * 123 FROM EMPLOYEE 124 WHERE EMPLOYEE.EMP_NO = 1001 </pre>

Uspešnim izvršavanjem upita na replici, potvrđeno je da je polje **SALARY** ažurirano na novu vrednost. Ovaj test demonstrira integritet transakcija – replikacioni *engine* nije samo prekopirao red, već je precizno izvršio UPDATE operaciju nad ciljnim zapisom na *slave* bazi zadržavajući potpunu konzistentnost podataka između dva čvora.

Primer 2: Brisanje podataka na master-u

Ovaj primer obuhvata operaciju brisanja (DELETE). Ovaj test je specifičan jer uključuje rad sa dve povezane tabele. Da bi se zaposleni obrisao, neophodno je prvo ukloniti sve zapise o njegovoj istoriji plata zbog stranih ključeva (*Foreign Keys*), a zatim i sam zapis o zaposlenom.

Izvorni kod: Brisanje na masteru
<pre> 125 DELETE FROM SALARY_HISTORY 126 WHERE EMP_NO = 1001; 127 -- COMMIT 128 DELETE FROM EMPLOYEE 129 WHERE EMP_NO = 1001; 130 -- COMMIT </pre>

Pošto su prethodne akcije prihvaćene, sada je potrebno da vidimo da li je preslikano novo stanje i na replikaciju

Izvorni kod: Provera brisanja na replikaciji

```
131 SELECT *
132 FROM EMPLOYEE
133 LEFT JOIN SALARY_HISTORY ON EMPLOYEE.EMP_NO =
    SALARY_HISTORY.EMP_NO
134 WHERE EMPLOYEE.EMP_NO = 1001
```

Rezultat upita na replici je prazan (0 redova) što je jasan dokaz da je replikacioni *engine* uspešno preneo obe **DELETE** operacije. Ovim je potvrđeno da sistem verno replicira kompletan životni ciklus podatka: od njegovog kreiranja (**INSERT**), preko modifikacije (**UPDATE**), pa sve do trajnog uklanjanja iz sistema (**DELETE**).

Posebno je značajno što je očuvan referencijalni integritet na obe baze, čime je dokazano da je sekundarni čvor u svakom trenutku konzistentna kopija primarnog sistema.

Primer 3: Statistički pregled publikovanih tabela

Ovaj upit daje detaljan uvid u broj tabela koje Firebird *engine* aktivno prati:

Izvorni kod: Provera brisanja na replikaciji

```
135 SELECT P.RDB$PUBLICATION_NAME AS PUBLIKACIJA,
136         COUNT(PT.RDB$TABLE_NAME) AS BR_TABELA
137 FROM RDB$PUBLICATIONS P
138 LEFT JOIN RDB$PUBLICATION_TABLES PT
139 ON PT.RDB$PUBLICATION_NAME = P.RDB$PUBLICATION_NAME
140 GROUP BY P.RDB$PUBLICATION_NAME;
```

Ovaj upit povezuje definiciju publikacije sa tabelom meta-podataka koja sadrži spisak svih uključenih objekata. Rezultat prikazuje ime aktivne publikacije i ukupan broj tabela iz EMPLOYEE modela koje su ušle u proces replikacije. Ovo je potvrda da je komanda INCLUDE ALL ispravno procesuirana i da nijedan segment baze nije izostavljen iz procesa sinhronizacije.

Primer 4: Health-check replikacije

Za stabilan rad replikacije nije dovoljno samo pratiti prenos podataka, već je neophodno kontrolisati i stanje transakcija. Ovaj upit omogućava uvid u „gap“ (razmak) između najstarije aktivne i sledeće planirane transakcije, što je direktan pokazatelj opterećenja i efikasnosti sistema.

Izvorni kod: Provera brisanja na replikaciji

```
141 SELECT
142     MON$NEXT_TRANSACTION           AS MASTER_NEXT_TID,
143     MON$OLDEST_ACTIVE             AS MASTER_OAT,
144     MON$OLDEST_TRANSACTION        AS MASTER_OIT,
145     MON$NEXT_TRANSACTION
146     - MON$OLDEST_ACTIVE           AS GC_GAP
147 FROM MON$DATABASE;
```

Razlika između vrednosti MON\$NEXT_TRANSACTION na masteru i replici predstavlja ključni indikator zdravlja celokupnog sistema. Mala razlika, u opsegu od 1 do 10, potvrđuje da replika prati master bazu gotovo u realnom vremenu i da se podaci uspešno sinhronizuju. Velika razlika ukazuje na tehnički zastoj u arhiviranju žurnala ili problem pri primeni segmenata na samu repliku. U takvim situacijama, svi neophodni detalji za dijagnostiku i rešavanje problema nalaze se u replication.log fajlu.

Primer 5: Isključivanje tabele iz publikacije

Ovaj primer prikazuje proces modifikacije obuhvata replikacije isključivanjem specifične tabele iz procesa sinhronizacije. Korišćenjem komande EXCLUDE TABLE, tabela SALARY_HISTORY se izuzima iz publikacije, što znači da se promene nad njom više neće prenositi na repliku, dok ostatak baze nastavlja sa radom. Provera kroz sistemsku tabelu RDB\$PUBLICATION_TABLES služi da potvrdi novo stanje i osigura da su u publikaciji ostale samo željene tabele.

Izvorni kod: Isključivanje tabele iz publikacije

```
148 ALTER DATABASE EXCLUDE TABLE SALARY_HISTORY FROM PUBLICATION;
149 SELECT RDB$TABLE_NAME
150 FROM RDB$PUBLICATION_TABLES
151 WHERE RDB$PUBLICATION_NAME = 'RDB$DEFAULT';
```

7. Zaključak i poređenje sa drugim sistemima

Iako je Firebird 4.0 (i noviji 5.0) doneo revolucionarne promene, važno je razumeti njegovu poziciju u odnosu na PostgreSQL i MS SQL Server, naročito u segmentu visoke dostupnosti:

Karakteristika	Firebird 4.0 +	PostgreSQL	MS SQL Server
Arhitektura	Native MGA (MVCC)	Native MVCC	Page-based (zaključavanje)
Replikacija	Native Binary & Logical	Streaming (Physical) & Logical	AlwaysOn / Log Shipping
Podešavanje	Srednje (tekstualni .conf)	Kompleksno (mnogo parametara)	Jednostavno (kroz SSMS GUI)
Hardverski zahtevi	Izuzetno niski (Low footprint)	Srednji do visoki	Visoki (zahteva dosta RAM-a)
Cena	Besplatno (Open Source)	Besplatno (Open Source)	Visoka (po jezgru procesora)

Tabela 2: Uporedne karakteristike Firebird-a i njegovih konkurenata

Na osnovu svega navedenog, može se zaključiti da implementacija replikacije u Firebird 4.0 sistemima ne predstavlja samo puko preslikavanje podataka sa jednog servera na drugi, već zahteva precizno razumevanje transakcione logike i unutrašnje arhitekture baze. Ključna prednost ovakvog pristupa leži u **MGA (Multi-Generational Architecture)** modelu, koji omogućava da proces sinhronizacije teče bez blokiranja korisnika i bez potrebe za kompleksnim lock-mehanizmima koji su karakteristični za Page-level sisteme poput MS SQL Servera. Dok konkurencija često zahteva dodatno licenciranje ili komplikovanje podešavanje mrežnih streaming protokola, Firebird nudi direktnu kontrolu nad replikacionim žurnalom uz minimalno opterećenje resursa servera.

Poseban fokus pri projektovanju ovakvih sistema mora biti stavljen na sprečavanje „echo” efekta, koji u Multi-Master konfiguracijama može dovesti do potpunog kolapsa procesora i prepunjavanja diska. Rešavanje ovog problema putem systemske funkcije RDB\$GET_CONTEXT i sesijskih varijabli unutar trigera predstavlja najefikasniji metod, jer se kontrola vrši na nivou same baze, a ne preko eksternih aplikacija. Na taj način se osigurava da svaka promena bude zabeležena samo jednom, čime se prekida lanac beskonačnih transakcija i omogućava stabilan rad baze u realnom vremenu. Na kraju, glavni zaključak je da pravilno podešena replikacija, pruža maksimalnu dostupnost podataka uz zadržavanje punog integriteta, što je ključni cilj svakog distribuiranog sistema.

8. Literatura

- [1] **Firebird 4.0 Release Notes**, Firebird Foundation (Zvanična dokumentacija o nativnoj replikaciji).
- [2] **Firebird 4.0 Language Reference**, poglavlja o `RDB$GET_CONTEXT` i `REPLICATING` varijablama.
- [3] **Havelund, K.:** "Database Replication Mechanisms", Journal of Software Engineering (za teorijski deo o CAP teoremi).
- [4] **Harrison, L.:** "Firebird Performance Tuning", odeljci o optimizaciji žurnala i disk I/O operacija.
- [5] **IBSurgeon Whitepapers:** Detaljne analize o oporavku baze i nbackup alatu.